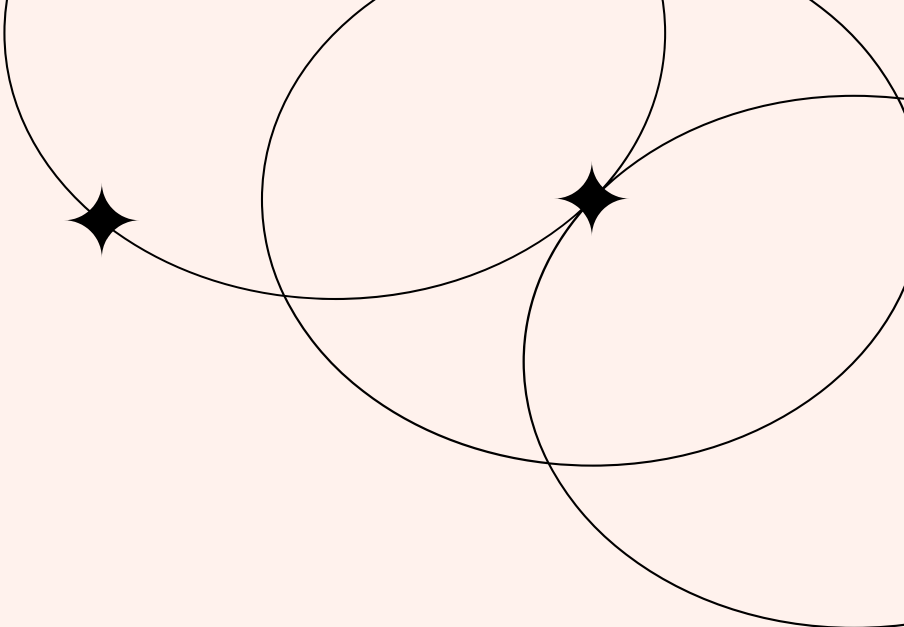
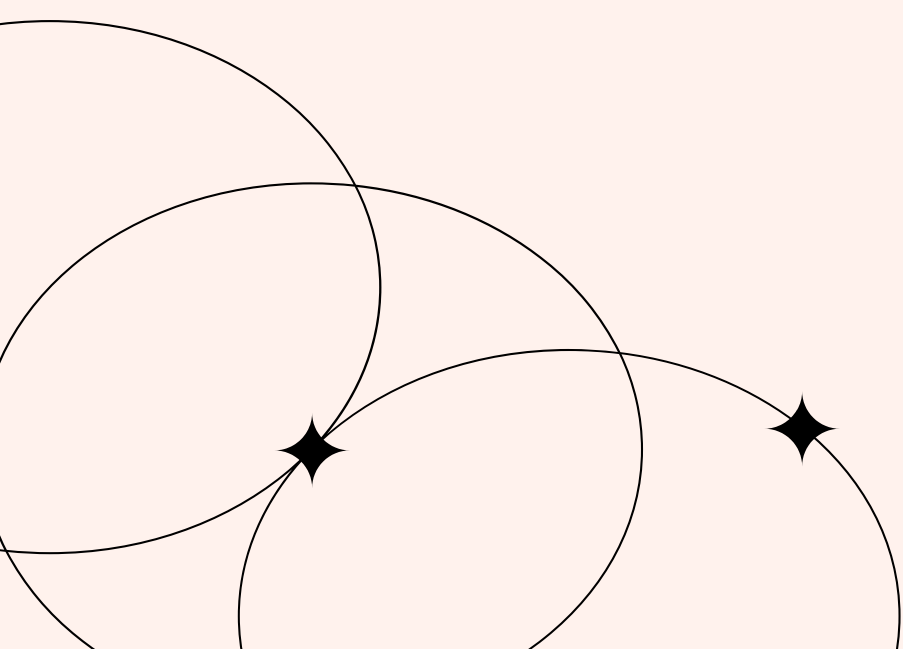




Storage engine i upravljanje podacima u PostgreSQL bazi podataka

Luka Veličković, 2101



Data Directory

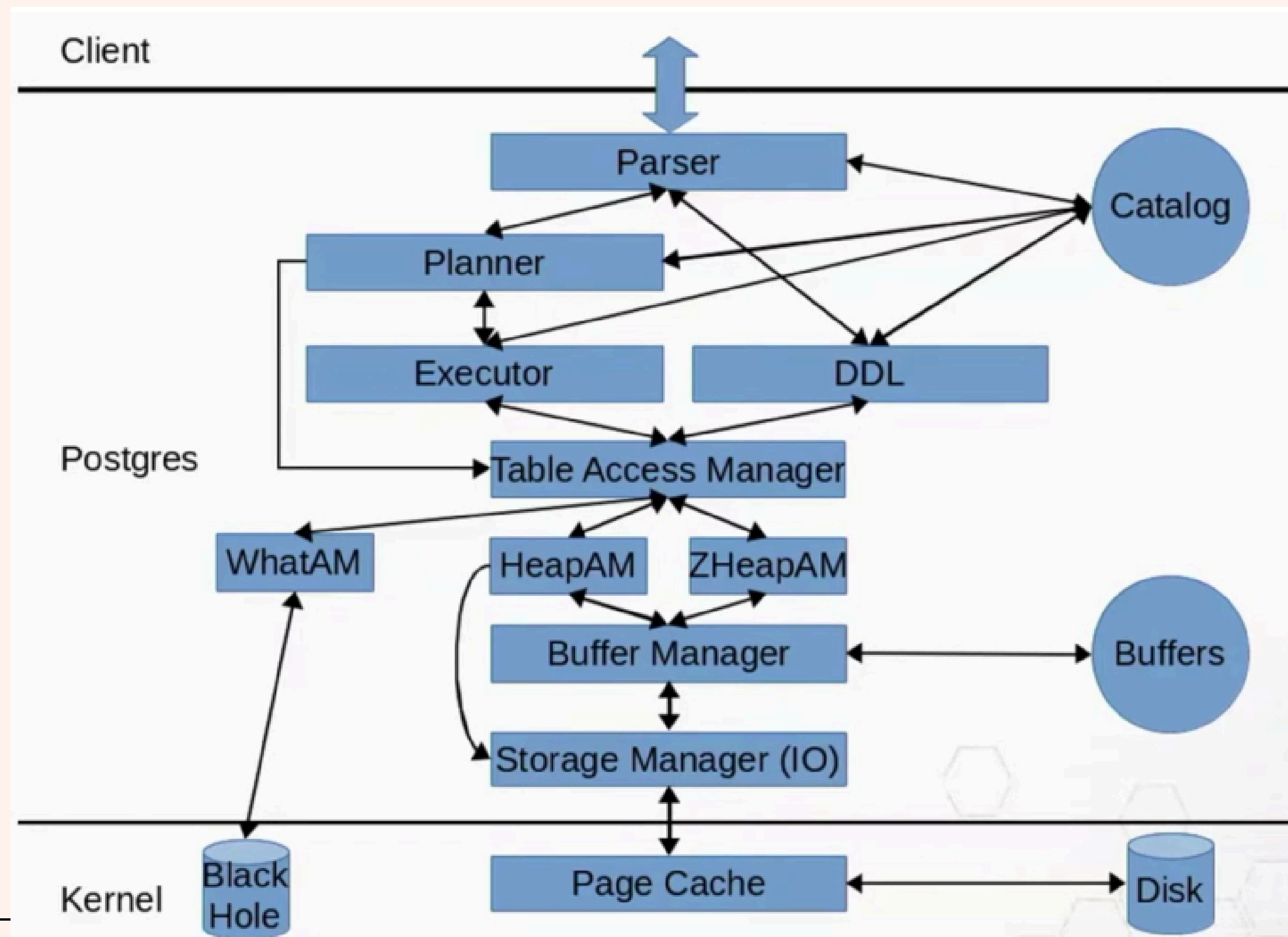
- fizička lokacija na mašini gde PostgreSQL skladišti svoje data fajlove

```
postgres=# show data_directory;
 data_directory
-----
/var/lib/postgresql/18/main
(1 row)
```

```
lule@lule-Asus-Vivobook:~$ sudo ls /var/lib/postgresql/18/main
base          pg_multixact  pg_stat       PG_VERSION    postmaster.pid
global        pg_notify     pg_stat_tmp   pg_wal
pg_commit_ts  pg_replslot   pg_subtrans   pg_xact
pg_dynshmem   pg_serial     pg_tblspc     postgresql.auto.conf
pg_logical    pg_snapshots  pg_twophase    postmaster.opts
```

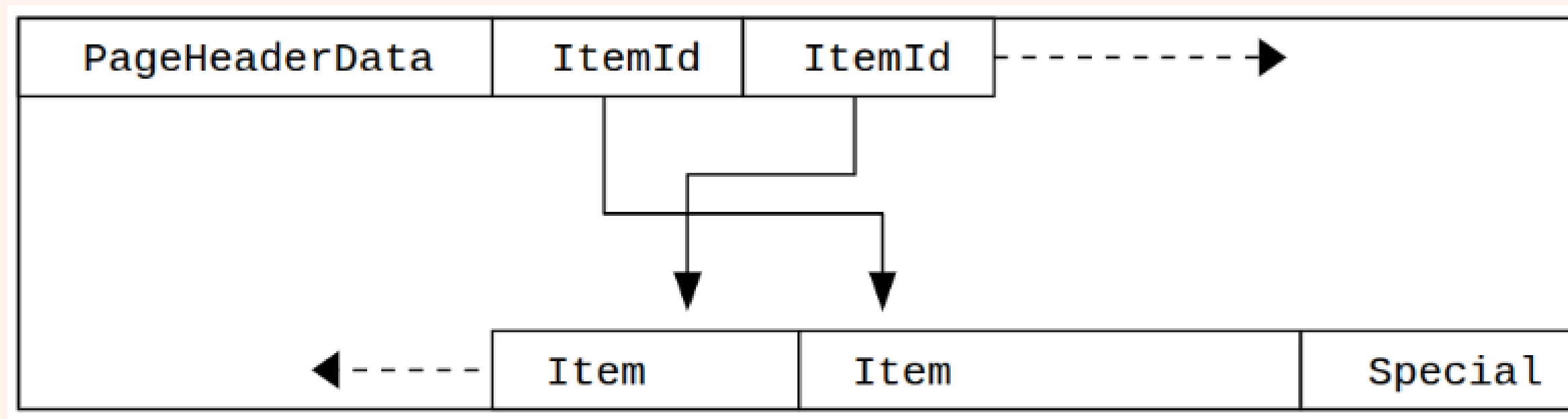
- base/ – kolekcija fajlova u kojima se korisnički podaci skladište. base/ direktorijum sadrži kolekciju direktorijuma čiji su nazivi OID baza (template0, template1, postgres, ...), a svaki poddirektorijum sadrži fajlove čiji je naziv OID objekta (relacije)
- global/ – zajednički objekti na nivou klastera (korisničke role, permisije...)
- pg_xact/ – status svih transakcija (in-progress, committed, rolled back)
- pg_wal/ – sadrži WAL segmente (fajlovi u koje se upisuju WAL zapisi)

PostgreSQL arhitektura



PostgreSQL stranice

- sve tabele i indeksi se skladište kao niz stranica fiksne dužine (8KB)



- header – 24 bajtova, sadrži podatke kao što su pokazivači lower i upper, checksum, kontrolni bitovi...
- ItemId – četvorobajtni pokazivač na redove (item), čuva dužinu reda i offset reda u stranici (i par kontrolnih bitova)
- Item – jedan red (tuple) iz tabele (ili ulaz u indeksu)
 - Svaki Item ima jedinstveni CTID (redni broj stranice, indeks ItemId-ja u toj stranici)
 - update dovodi do upisa novog reda u stranici
- Special – polje za slobodnu upotrebu, npr. pokazivač na susedni čvor kod indeksa

PostgreSQL stranice

```
create extension pageinspect;

create database demo;

create table app_user (
    id          int primary key,
    name        varchar(20) not null,
    favorite_animal text
);
```

- insert into app_user values (1, 'Luka', 'zebra');

```
lule@lule-Asus-Vivobook:~$ sudo hexdump -C /var/lib/postgresql/18/main/base/16454/16510
00000000  00 00 00 00 10 99 d2 01  e5 d9 00 00 1c 00 d8 1f  |.....|
00000010  00 20 04 20 00 00 00 00  d8 9f 4e 00 00 00 00 00  |. . . . .N.....|
00000020  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |.....|
*
00001fd0  00 00 00 00 00 00 00 00  09 03 00 00 00 00 00 00  |.....|
00001fe0  00 00 00 00 00 00 00 00  01 00 03 00 02 08 18 00  |.....|
00001ff0  01 00 00 00 0b 4c 75 6b  61 0d 7a 65 62 72 61 00  |.....Luka.zebra.|
00002000
```

```
demo=# select * from page_header(get_raw_page('app_user', 0));
   lsn   | checksum | flags | lower | upper | special | pagesize | version | prune_xid
-----+-----+-----+-----+-----+-----+-----+-----+-----
 0/1D29910 |          0 |      0 |    28 |   8152 |       8192 |      8192 |        4 |          0
(1 row)
```

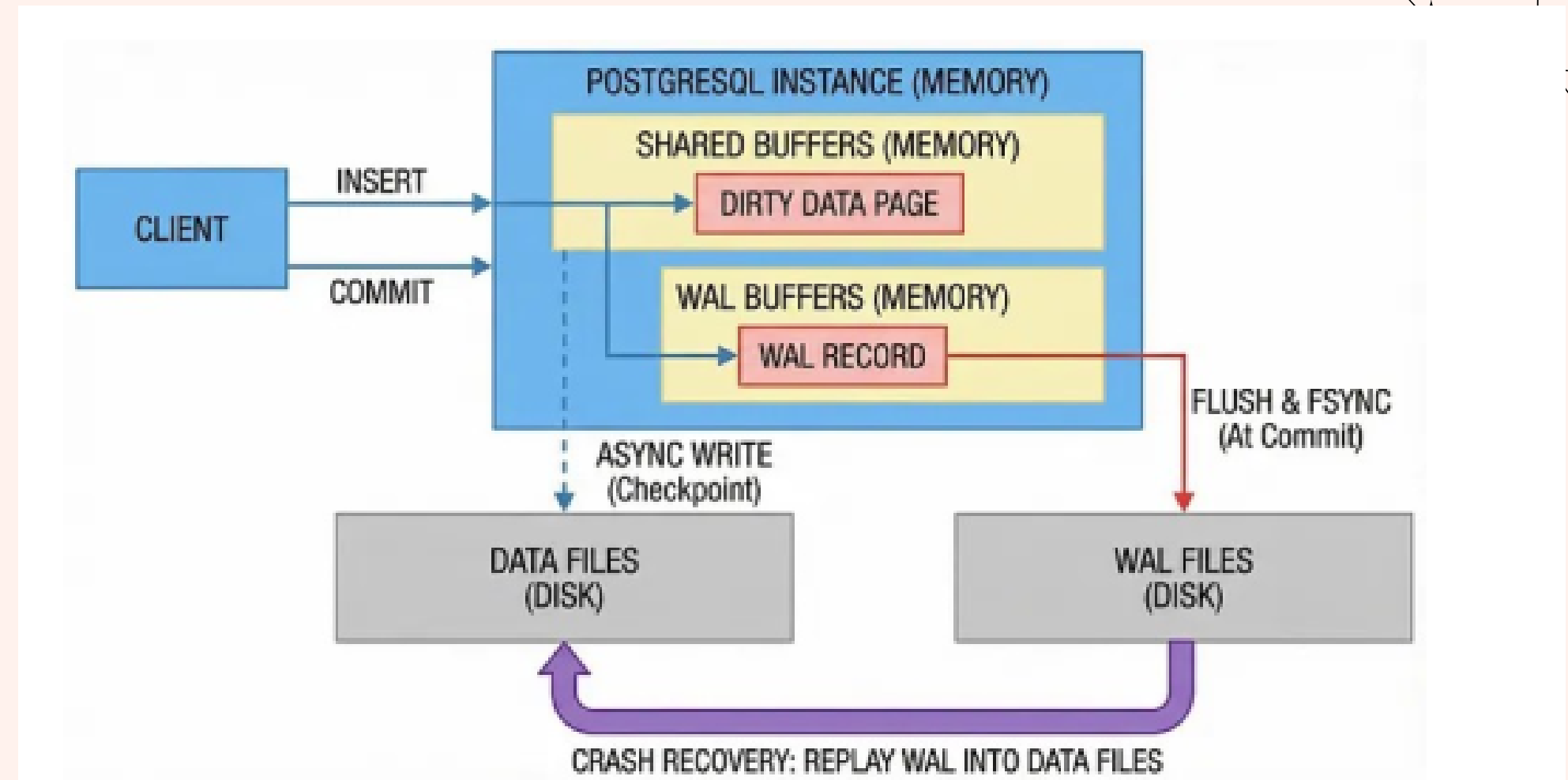
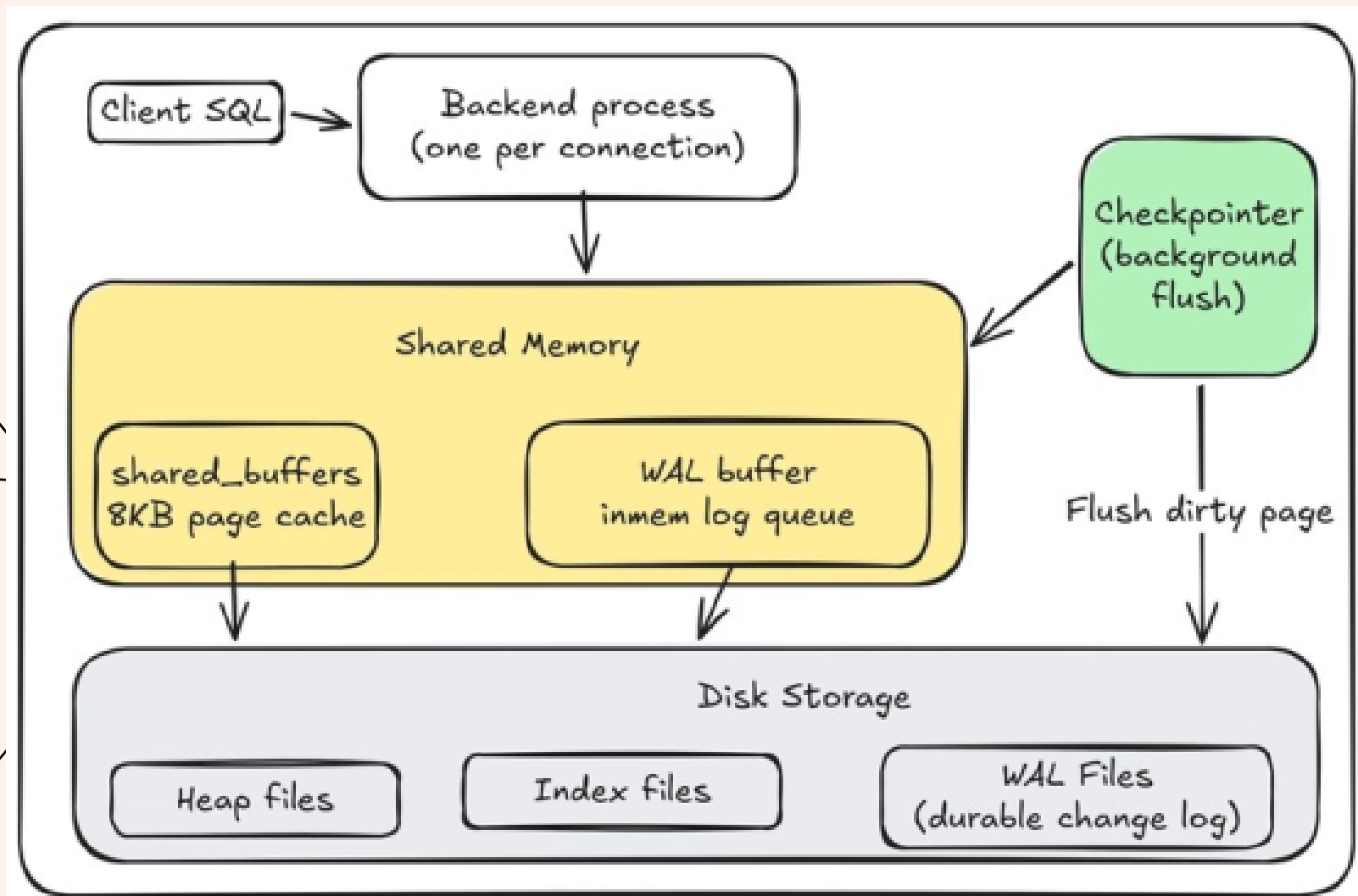

PostgreSQL stranice

- update app_user set favorite_animal = 'lion' where id = 1;

```
lule@lule-Asus-Vivobook:~$ sudo hexdump -C /var/lib/postgresql/18/main/base/16454/16510
00000000  00 00 00 00 80 d4 d5 01 4e b8 00 00 20 00 b0 1f |.....N... ..|
00000010  00 20 04 20 0a 03 00 00 d8 9f 4e 00 b0 9f 4e 00 |. . ....N...N.|
00000020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00001fb0  0a 03 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00001fc0  02 00 03 80 02 29 18 00 01 00 00 00 0b 4c 75 6b |.....).....Luk|
00001fd0  61 0d 70 61 6e 64 61 00 09 03 00 00 0a 03 00 00 |a.panda.....|
00001fe0  00 00 00 00 00 00 00 00 02 00 03 40 02 05 18 00 |.....@....|
00001ff0  01 00 00 00 0b 4c 75 6b 61 0d 7a 65 62 72 61 00 |.....Luka.zebra.|
00002000
```

```
demo=# SELECT t_ctid, raw_flags, combined_flags
        FROM heap_page_items(get_raw_page('app_user', 0)),
        LATERAL heap_tuple_infomask_flags(t_infomask, t_infomask2)
        WHERE t_infomask IS NOT NULL OR t_infomask2 IS NOT NULL;
 t_ctid |                               raw_flags                               | combined_flags
-----+-----+-----
(0,2)  | {HEAP_HASVARWIDTH,HEAP_XMIN_COMMITTED,HEAP_XMAX_COMMITTED,HEAP_HOT_UPDATED} | {}
(0,2)  | {HEAP_HASVARWIDTH,HEAP_XMIN_COMMITTED,HEAP_XMAX_INVALID,HEAP_UPDATED,HEAP_ONLY_TUPLE} | {}
(2 rows)
```

PostgreSQL write i read path



- Kada se klijentu potvrđuje commit?
 - fsync
 - fdatasync
 - asinhrono

PostgreSQL write i read path

```
demo=# explain (analyze, buffers) insert into app_user values (1, 'Luka', 'zebra');
               QUERY PLAN
-----
Insert on app_user  (cost=0.00..0.01 rows=0 width=0) (actual time=0.305..0.305 rows=0.00 loops=1)
  Buffers: shared hit=14 read=1 dirtied=3 written=2
  -> Result  (cost=0.00..0.01 rows=1 width=94) (actual time=0.002..0.003 rows=1.00 loops=1)
Planning Time: 0.046 ms
Execution Time: 0.326 ms
(5 rows)
```

```
lule@lule-Asus-Vivobook:~$ sudo hexdump -C /var/lib/postgresql/18/main/base/16454/16588
00000000  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |.....|
*
00002000
```

```
lule@lule-Asus-Vivobook:~$ sudo /usr/lib/postgresql/18/bin/pg_waldump /var/lib/p
ostgresql/18/main/pg_wal/00000001000000000000000001 --relation=1663/16454/16588

rmgr: Heap          len (rec/tot):      70/    70, tx:          837, lsn: 0/01FB1F30,
  prev 0/01FB1EF8, desc: INSERT+INIT off: 1, flags: 0x00, blkref #0: rel 1663/164
54/16588 blk 0
```


PostgreSQL write i read path

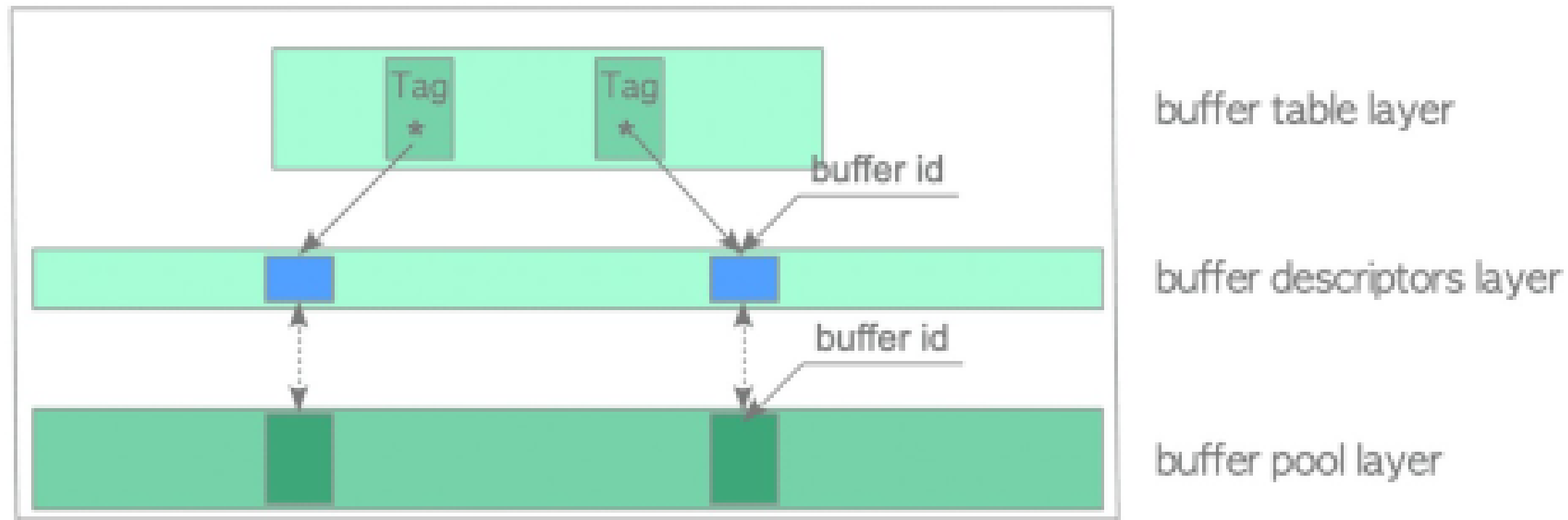
```
demo=# explain (analyze, buffers) select * from app_user where id = 2300;  
QUERY PLAN
```

```
-----  
Index Scan using app_user_pkey on app_user  (cost=0.29..8.30 rows=1 width=21) (actual time=0.032..0.033 rows=1.00 loops=1)  
  Index Cond: (id = 2300)  
  Index Searches: 1  
  Buffers: shared read=3  
Planning:  
  Buffers: shared hit=40 read=22  
Planning Time: 1.006 ms  
Execution Time: 0.124 ms  
(8 rows)
```

```
demo=# explain (analyze, buffers) select * from app_user where id = 2300;  
QUERY PLAN
```

```
-----  
Index Scan using app_user_pkey on app_user  (cost=0.29..8.30 rows=1 width=21) (actual time=0.022..0.024 rows=1.00 loops=1)  
  Index Cond: (id = 2300)  
  Index Searches: 1  
  Buffers: shared hit=3  
Planning Time: 0.095 ms  
Execution Time: 0.064 ms  
(6 rows)
```

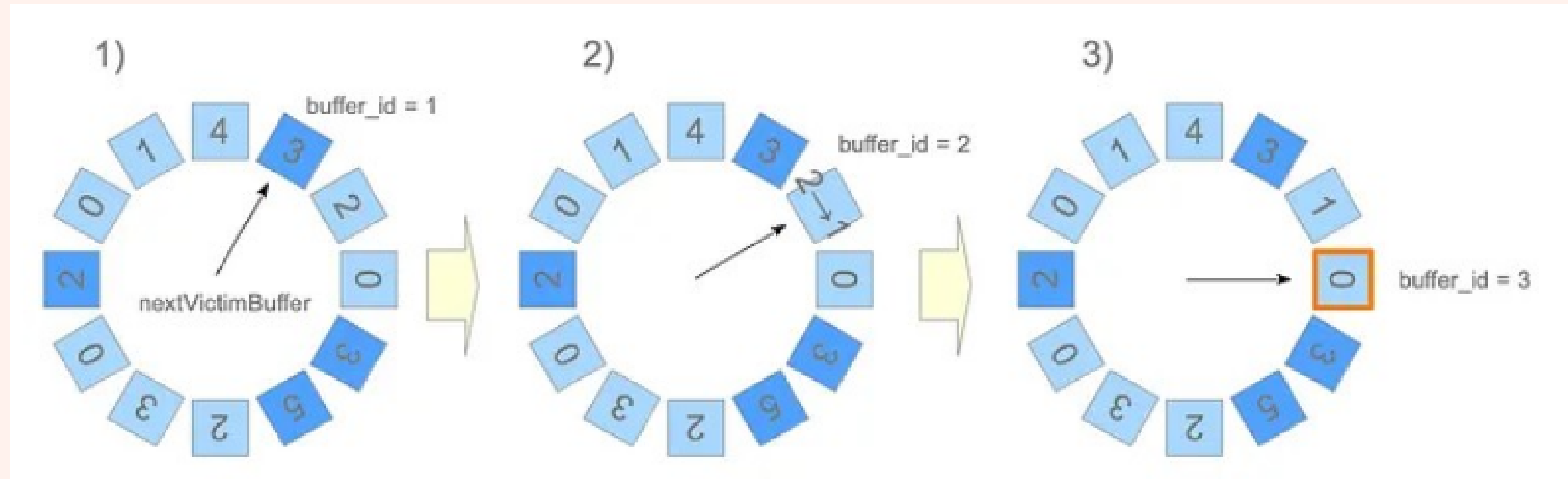
PostgreSQL upravljanje baferima



```
typedef struct buftag
{
    Oid          spcOid;
    Oid          dbOid;
    RelFileNumber relNumber;
    ForkNumber   forkNum;
    BlockNumber  blockNum;
} BufferTag;
```

- buffer pool – niz koji skladišti stranice. Svakom slotu se pristupa na osnovu buffer_id-ja
- buffer descriptors – svaki deskriptor ima odgovarajuću stranicu u bufer pool layer-u i sadrži metapodatke o njoj
 - refcount – broj procesa koji trenutno pristupaju stranici iz pool-a. Kada proces pristupi stranici inkrementira se, kada završi pristup dekrementira se
 - usage_count – broj puta koliko je stranici pristupljeno od kako je učitana u bafere
 - content_lock – lock koji kontroliše pristup povezanoj stranici u pool-u
- buffer table layer – hash tabela koja čuva mapiranje između buffer tag-a i buffer_id – ja

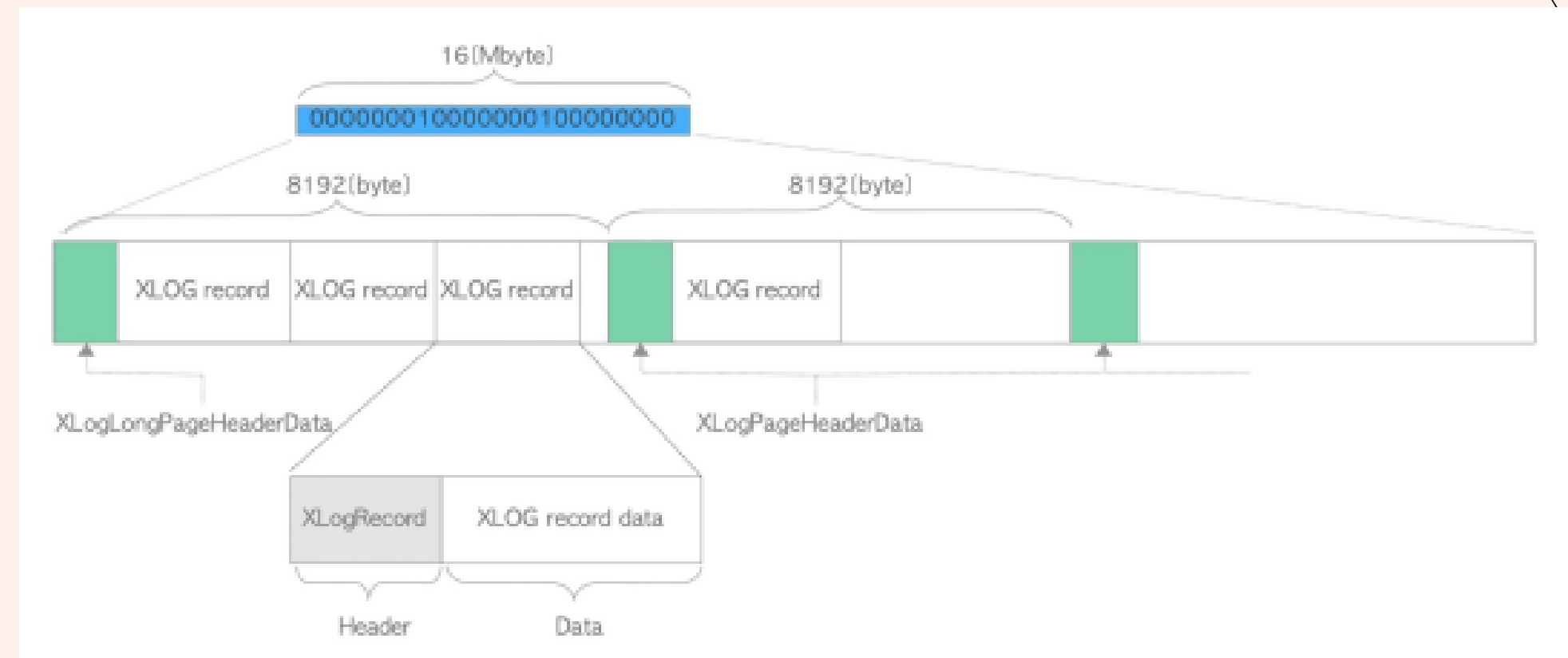
PostgreSQL upravljanje baferima



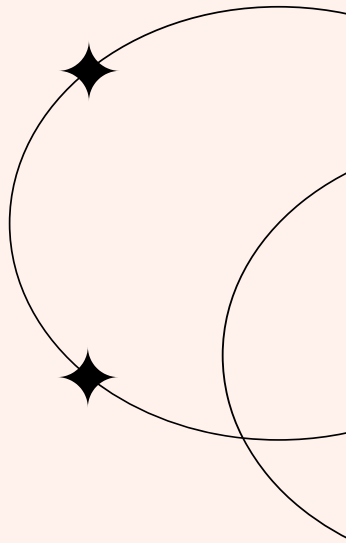
- Kada je shared buffers pun, potrebno je odrediti koja stranica će biti žrtvovana kako bi se oslobodilo mesto za novu. U tu svrhu Postgres koristi clock sweep algoritam
 - Postoji pokazivač `nextVictimBuffer` koji pokazuje na element u buffer descriptor nizu
 - Koristi se `usageCount` vrednost elementa u descriptor nizu
 - Ako je `usageCount = 0`, ta stranica se žrtvuje
 - U suprotnom, `usageCount` se dekrementira i `nextVictimBuffer` se uvećava za 1

PostgreSQL WAL

- WAL segment – 1 WAL fajl u pg_wal direktorijumu
 - Podrazumevano 16MB
 - Imena su u formatu
00000001 00000000 00000001,
00000001 00000000 00000002....
- XLOG record – 1 zapis u WAL-u
- LSN – Log Sequence Number, lokacija gde je XLOG record upisan u transakcioni log, jedinstveni identifikator WAL zapisa
- REDO point – tačka od koje se replay-uje WAL nakon recovery-ja
 - upisuje se u wal prilikom checkpoint akcije
- Full page zapisi – WAL record koji sadrži sadržaj cele stranice



PostgreSQL WAL

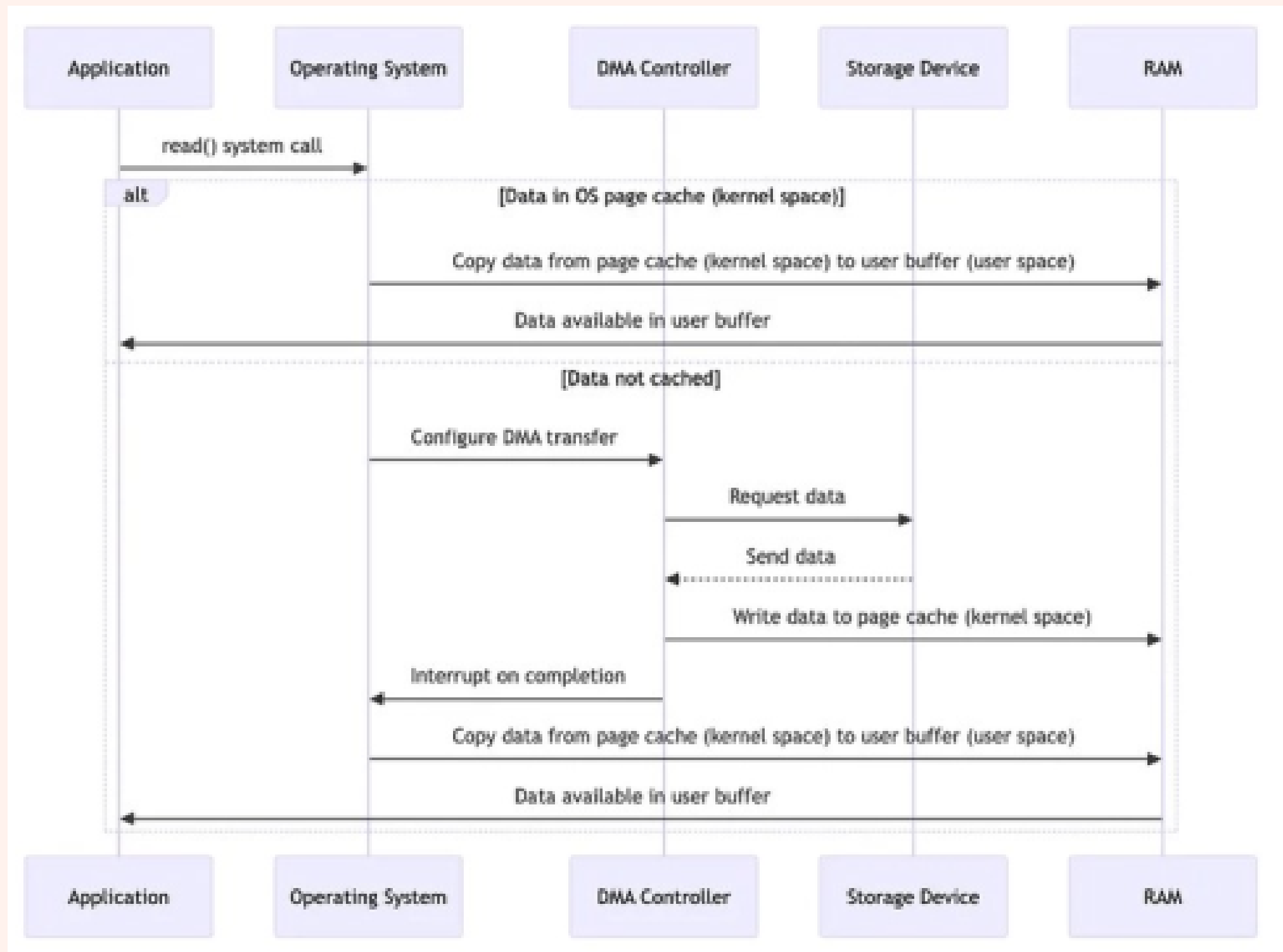


```
create table wal_test (  
    id bigint primary key,  
    payload text  
);  
  
insert into wal_test  
select g, repeat('x', 2000)  
from generate_series(1, 200000) as g;
```

```
update wal_test  
set payload = repeat('y', 2000)  
where id % 2 = 0;
```

Checkpoint ponašanje	full_page_write	Količina generisanog WAL-a	Broj fsync poziva
Nakon svakog update-a	On	204.05 MB	78
Jedan checkpoint nakon svih update-a	On	134.83 MB	19
Nakon svakog update-a	Off	119.75 MB	43
Jedan checkpoint nakon svih update-a	Off	115.48 MB	7

PostgreSQL – asinhroni IO



Tradicionálni pristup IO

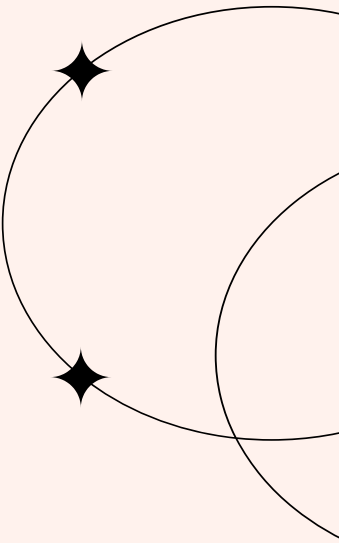
- Tradicionalni pristup – podaci se keširaju u kešu operativnog sistema.
- Ako stranica koja se zahteva nije u kešu, OS izdaje zahtev za njeno pribavljanje.
- Upisuje se u keš, a zatim u memoriju korisničkog procesa
- Korisnički proces je blokiran tokom ovog procesa
- Asinhroni IO – korisnički proces se ne blokira prilikom pribavljanja stranice
- Direktni IO – zaobilazi se keš OS-a, a stranica se direktno kopira u korisnički prostor, u ovom slučaju shared buffers PostgreSQL-a

PostgreSQL – asinhroni IO

- PostgreSQL nudi podešavanje `io_method`, kojim se određuje sinhronost read operacija
 - `io_uring`
 - `worker`
 - `sync`
- Asinhroni IO donosi prednosti kada su operacije disk (IO) heavy, a ne CPU heavy

```
create table app_user (  
    id          serial primary key,  
    name        varchar(20) not null,  
    password    text,  
    registration_date timestamp default now()  
);
```

```
insert into app_user (name, password)  
select  
    'Name' || i,  
    md5(random()::text)  
from generate_series(1, 10000000) as g(i);
```



PostgreSQL – asinhroni IO

```
1) select count(*), max(registration_date) from app_user where id > 500;  
2) select id, repeat(md5(password), 3) from app_user where id > 2500000;
```

io_method	Upit 1 – cold read	Upit 1 – warm read	Upit 2 - cold read	Upit 2 – warm read
io_uring	929 ms	595.71 ms	7.8 s	7.62 s
worker	1055 ms	527.34 ms	7.96 s	7.53 s
sync	1350 ms	614.24 ms	7.81 s	7.28 s

io_method	Upit	Cache hits	Cache misses	I/O blocking time
io_uring	Upit 1 – cold read	0	112_582	153.38 ms
	Upit 1 – warm read	846	111_736	136.2 ms
worker	Upit 1 – cold read	0	112_582	676 ms
	Upit 1 – warm read	1128	114_454	5.93 ms
sync	Upit 1 – cold read	0	112_582	1574.76 ms
	Upit 1 - warm read	564	112_018	283.46 ms

